TADs e algoritmos em grafos

Relatório técnico dos trabalhos práticos

Ayrton Sanabio - Matrícula 0117925
Estruturas de Dados 2 - Professor Walace de Almeida Rodrigues
Versão de 19 de outubro de 2026

1. Introdução

O trabalho implementa bibliotecas em C para pilha, fila, lista duplamente encadeada e grafo estático. Sobre a TAD de grafo são executadas busca em largura, busca em profundidade, caminho mínimo, componentes conexos e árvore geradora mínima. A entrada e a saída são textuais, com carregamento de grafos por arquivo.

A implementação é dividida em interfaces e módulos independentes. O programa principal valida as entradas e seleciona a operação; os algoritmos utilizam apenas as funções públicas da TAD. Não acessam diretamente seus campos internos.

2. Estruturas e módulos

Módulo	Responsabilidade
01-pilhas / 02-filas	Nós encadeados, políticas LIFO/FIFO, contagem e demonstração.
03-listas	Nós com anterior e próximo; inserção, consulta e remoção por posição.
04-grafos/grafo	Vetores fixos de vértices e arestas, estrelas e persistência.
04-grafos/caminho	Dijkstra compartilhado e formato de saída do TP 1.
04-grafos/algoritmos	Percursos, componentes, Kruskal e saídas do TP 2.
05-arvores (adicional)	ABB completa, remoções, altura e três percursos.

O grafo reserva vetores de vértices e arestas na criação. Cada elemento tem ID e marca de atividade. Uma aresta armazena alfa, ômega e elos inteiros para as estrelas de entrada, saída e incidência. As listas de IDs de cada estrela são mantidas em ordem crescente.

```
Grafo
  +-- vertices[id]: ativo, inicio/fim das estrelas
  +-- arestas[id]: ativa, alfa, omega, elos das estrelas

Algoritmos -> interface grafo.h -> armazenamento interno
```

O índice zero significa ausência. Laços e paralelas são preservados com IDs próprios. Um laço aparece uma vez na estrela de incidência e contribui duas unidades ao grau total. A orientação depende das consultas realizadas.

3. Decisões de implementação

Percursos. BFS utiliza uma fila em vetor e marca o vértice na descoberta. DFS usa uma pilha explícita de quadros com o vértice atual e a próxima aresta. Isso reproduz a DFS recursiva sem depender da profundidade da pilha do processo. Vizinhos são explorados por ID de aresta crescente.

Caminho mínimo. Dijkstra mantém distância, predecessor e aresta predecessora. Seleciona linearmente o vértice aberto de menor distância. Pesos são externos à TAD, em float; a soma das distâncias usa double. Pesos negativos ou não finitos são rejeitados. No TP 1, a ausência de caminho imprime a partida; no TP 2, a mensagem é Sem caminho.

AGM. Kruskal ordena por peso, desempata pelo ID e usa Union-Find com compressão de caminhos e união por tamanho. Arestas que uniriam vértices do mesmo componente são descartadas. Pesos negativos finitos são permitidos. Em grafo desconexo, é informada a ausência de AGM para o conjunto completo.

Componentes. A busca é reiniciada em cada vértice não marcado. A função consulta incidências sem considerar a orientação; portanto, em dígrafos, obtém componentes fracamente conexos. Vértices isolados formam componentes próprios.

Remoções. Remover uma aresta atualiza suas listas de incidência. Remover um vértice remove primeiro suas arestas. Os IDs restantes ficam estáveis e os removidos não são reciclados. Ao salvar, o arquivo original exige compactação dos IDs; essa renumeração não modifica o grafo em memória.

4. Custos

V e E são quantidades ativas; Cv e Ce são capacidades reservadas. Com lacunas de remoção, uma varredura global de IDs pode percorrer a capacidade. Os custos abaixo correspondem ao código entregue.

Operação	Tempo
Consulta por ID / extremos / criar elemento	O(1)
Remover aresta / remover vértice	O(1) / O(grau do vértice)
Próxima aresta de uma estrela, partindo de membro	O(1)
BFS/DFS completa / componentes	O(Cv + E)
Dijkstra com seleção linear	O(V * Cv + E + Ce)
Kruskal, com ordenação usual	O(Cv + Ce + E log E)

A TAD ocupa O(Cv + Ce). Percursos e Dijkstra usam O(Cv) auxiliar; Kruskal usa O(Cv + E). Com capacidades proporcionais ao tamanho e sem lacunas, os custos usuais são O(V + E), O(V^2 + E) e O(V + E log E), respectivamente.

5. Validação

Os cinco programas foram compilados com GCC 6.3.0, C11 e as opções -Wall, -Wextra, -Wpedantic e -Werror. A execução em 19/10/2026 terminou sem avisos de compilação e com os testes aprovados.

Verificação	Critério utilizado
TADs básicas	Ordem LIFO/FIFO, consulta, tamanho, extremos, remoção até vazio e reinserção.
TAD de grafo	31 operações, capacidades, IDs, estrelas, laços, paralelas, arquivos e remoções.
540 estados de remoção	Graus e estrelas comparados com um modelo independente em C.
16 grupos de regressão	Saídas, formatos, menu, direção, desconexão, percursos e erros.
25 grafos aleatórios	BFS/DFS de referência; Bellman-Ford para distâncias; enumeração para AGM.
DFS com 4.000 vértices	Cadeia completa sem recursão na implementação.
ABB adicional	1.500 estados aleatórios, 120 estados de menu e cadeia de 3.000 nós.

Foram incluídos grafo vazio, vértice isolado, origem igual ao destino, laços, arestas paralelas, pesos zero, pesos negativos, arquivo truncado, conteúdo extra, extremos inexistentes e argumentos inválidos. No caminho mínimo, cada aresta impressa também é confrontada com os vértices e com a soma dos pesos.

Exemplo reproduzível

O arquivo exemplos/grafo.txt contém seis vértices e oito arestas. Os pesos correspondentes estão em exemplos/pesos.txt. O menor caminho de 1 a 6 custa 8; a AGM possui cinco arestas e custo total 9.

```
BFS: v1,a1,v2,a2,v3,a3,v4,a6,v5,a8,v6
DFS: v1,a1,v2,a3,v4,a5,v5,a8,v6,a2,v3

Caminho: v1,a2,v3,a4,v4,a5,v5,a8,v6
Distancia: 8

AGM: a2,a3,a4,a8,a5
```

No exemplo do TP 1, a partida 3 e a chegada 2 produzem 3 -> 6 -> 1 -> 2. Quando a partida é 1 e a chegada é 4, a saída contém somente 1, como determinado pelo enunciado.

Os testes são determinísticos e podem ser repetidos com testar.ps1. Não simulam falta de memória e não demonstram correção para todos os grafos possíveis; fornecem evidências de funcionamento e regressão dos contratos.

6. Uso, convenções e limites

O grafo é lido em texto: primeira linha com o número de vértices e arestas, seguida por uma linha origem destino para cada aresta. Cada linha contém exatamente dois inteiros. O arquivo opcional de pesos contém um número finito por aresta, na mesma ordem. Sem esse arquivo, cada peso vale 1.

```
.\compilar.ps1
.\testar.ps1

$g = 'exemplos/grafo.txt'
$p = 'exemplos/pesos.txt'
.\output\grafos.exe $g bfs 1
.\output\grafos.exe $g componentes
.\output\grafos.exe $g caminho 1 6 --pesos $p
.\output\grafos.exe $g agm --pesos $p
```

Nas buscas e no caminho do TP 2, --nao-dirigido permite considerar os dois sentidos. AGM e componentes sempre usam essa interpretação. A saída dos percursos registra a descoberta de cada vértice e a aresta que o descobriu; não registra retornos nem todas as arestas examinadas.

As assinaturas originais do TP 2 não recebem pesos. Foram preservadas como operações de peso unitário e complementadas por variantes ComPesos. As rotinas da TAD retornam zero ou NULL em erro. O terminal usa código 2 para entrada inválida; ausência de caminho ou de AGM é tratada como resultado normal.

IDs não reciclados podem esgotar a capacidade histórica mesmo depois de remoções. Salvar compacta IDs e persiste somente topologia: pesos externos precisam acompanhar a nova ordem. Essas escolhas cobrem pontos não especificados nas remoções adicionais e estão detalhadas no manual técnico.

7. Conclusão

A separação entre TAD, algoritmos e entrada/saída permite testar contratos isoladamente e reutilizar o armazenamento em tarefas distintas. As verificações mostraram a importância de tratar IDs, laços, paralelas, direção e estados vazios antes de analisar o custo dos algoritmos. A implementação oferece uma base concreta para explicar percursos, conjuntos disjuntos e caminhos mínimos.

Código e documentação foram concluídos com assistência de IA. Este texto relata decisões e validações técnicas; a avaliação pessoal da experiência e a preparação para a apresentação oral devem ser feitas pelo aluno. A pasta adicional 05-arvores foi concluída a pedido, com as 14 operações da ABB. A altura mede arestas: vazia = -1 e folha = 0. Busca, inserção e remoção custam O(h); altura e percursos iterativos custam O(n) em tempo e espaço auxiliar. Não há balanceamento automático. Os módulos originais de pilha, fila e lista permanecem preservados.

8. Referências

RODRIGUES, Walace de Almeida. *TP - TADs basicas.pdf*; *TP - TAD Grafo estatico.pdf*, especialmente pp. 3-10; *TP - algoritmos.pdf*. Enunciados de Estruturas de Dados 2. Publicação complementar no Classroom: remoção de vértices e arestas.